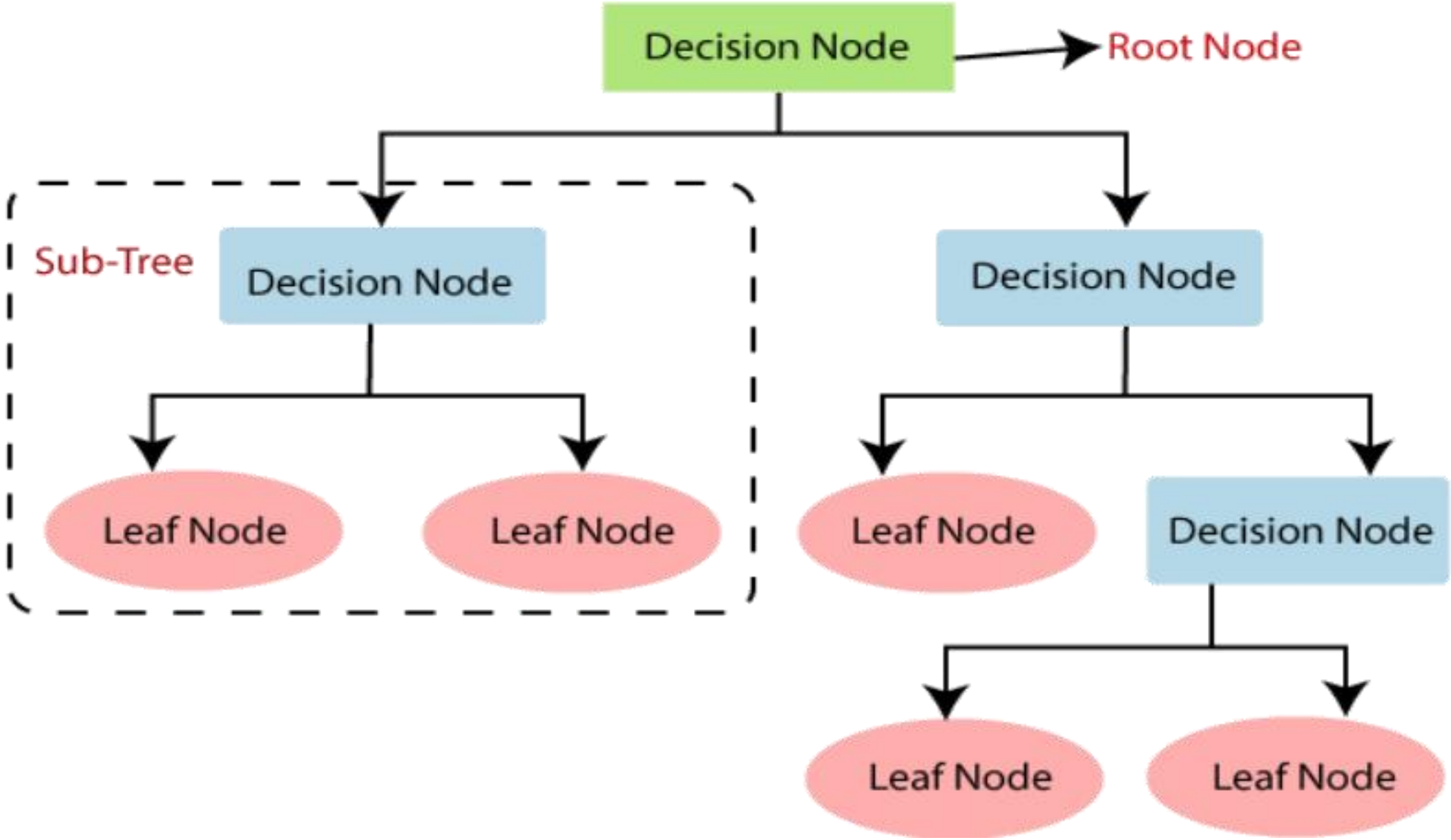
Decision Tree Classification Algorithm

- Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.
- It is a tree-structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.
- In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.

- It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.
- In order to build a tree, we use the CART algorithm, which stands for Classification and Regression Tree algorithm.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:


Decision Node
Root Node
Sub-Tree
Decision Node
Decision Node
Leaf Node
Leaf Node
Leaf Node
Decision Node
Leaf Node
Leaf Node

Why use Decision Trees?

- There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

  - Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
  - The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

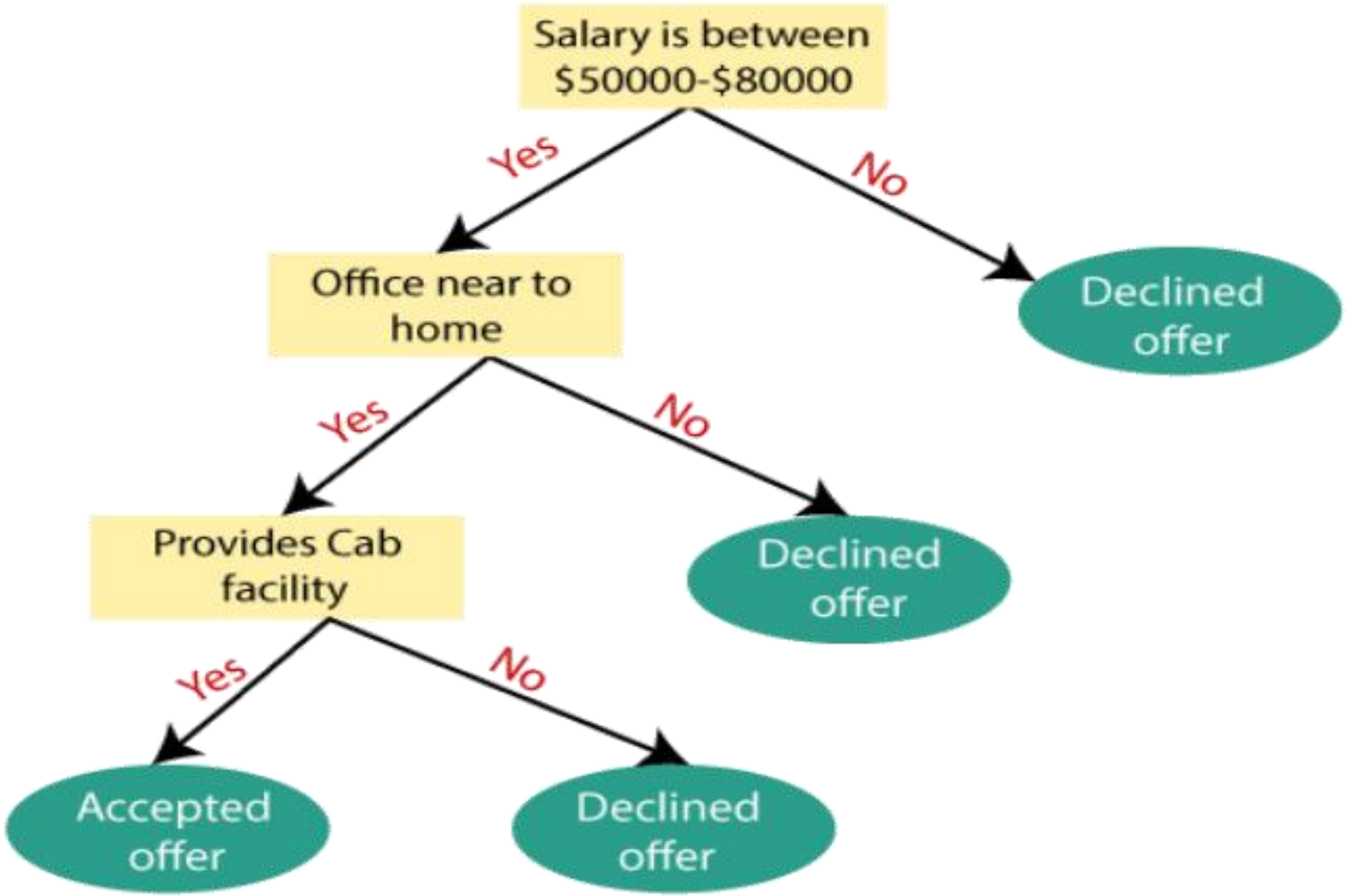
- Root Node: Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- Leaf Node: Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- Splitting: Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- Branch/Sub Tree: A tree formed by splitting the tree.
- Pruning: Pruning is the process of removing the unwanted branches from the tree.
- Parent/Child node: The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm:

- Step-1: Begin the tree with the root node, says S, which contains the complete dataset.
- Step-2: Find the best attribute in the dataset using Attribute Selection Measure (ASM).
- Step-3: Divide the S into subsets that contains possible values for the best attributes.
- Step-4: Generate the decision tree node, which contains the best attribute.
- Step-5: Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

- Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM).
- The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:


Salary is between $50000-$80000
Yes
No
Office near to home
Declined offer
Yes
No
Provides Cab facility
Declined offer
Yes
No
Accepted offer
Declined offer

Attribute Selection Measures

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as Attribute selection measure or ASM. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are:
- Information Gain
- Gini Index

Information Gain:

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:
- Information Gain= Entropy(S)-[(Weighted Avg) *Entropy(each feature)

- Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:
- Entropy(s)= -P(yes)log2 P(yes)- P(no) log2 P(no)

Where,

S= Total number of samples

P(yes)= probability of yes

P(no)= probability of no

Gini Index:

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:
- Gini Index= 1- ∑jPj2

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- oThe decision tree contains lots of layers, which makes it complex.
- oIt may have an overfitting issue, which can be resolved using the Random Forest algorithm.
- oFor more class labels, the computational complexity of the decision tree may increase.

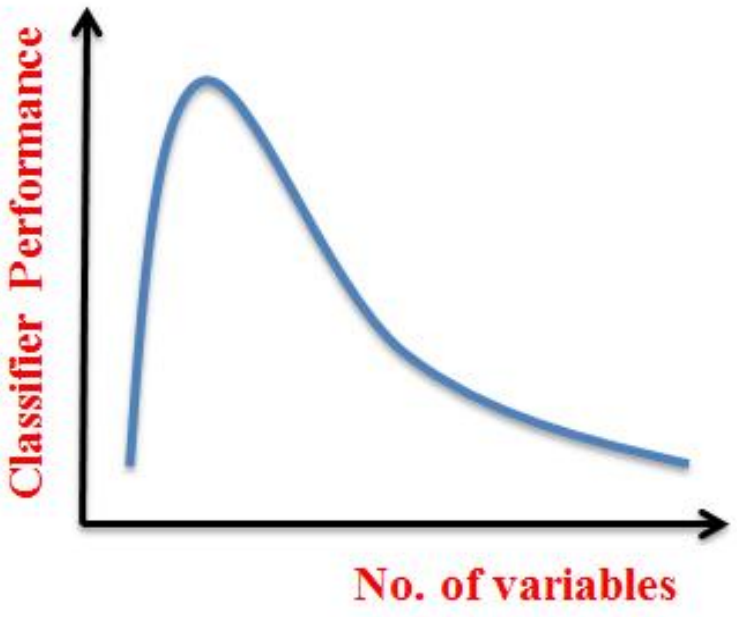
Feature Selection

Feature Selection

- It is the process of selecting a subset of relevant features for use in model construction.
- It is also called variable selection or attribute selection.
- It reduces the complexity of model.
- It either improve or maintain accuracy of model.

Curse of Dimensionality

- The amount of data required to achieve the same level of accuracy increases exponentially as the number of features increases.
- But in practice, the volume of training data available to us is fixed. Therefore, in most cases, the performance of the classifier will decrease with an increased number of variables.

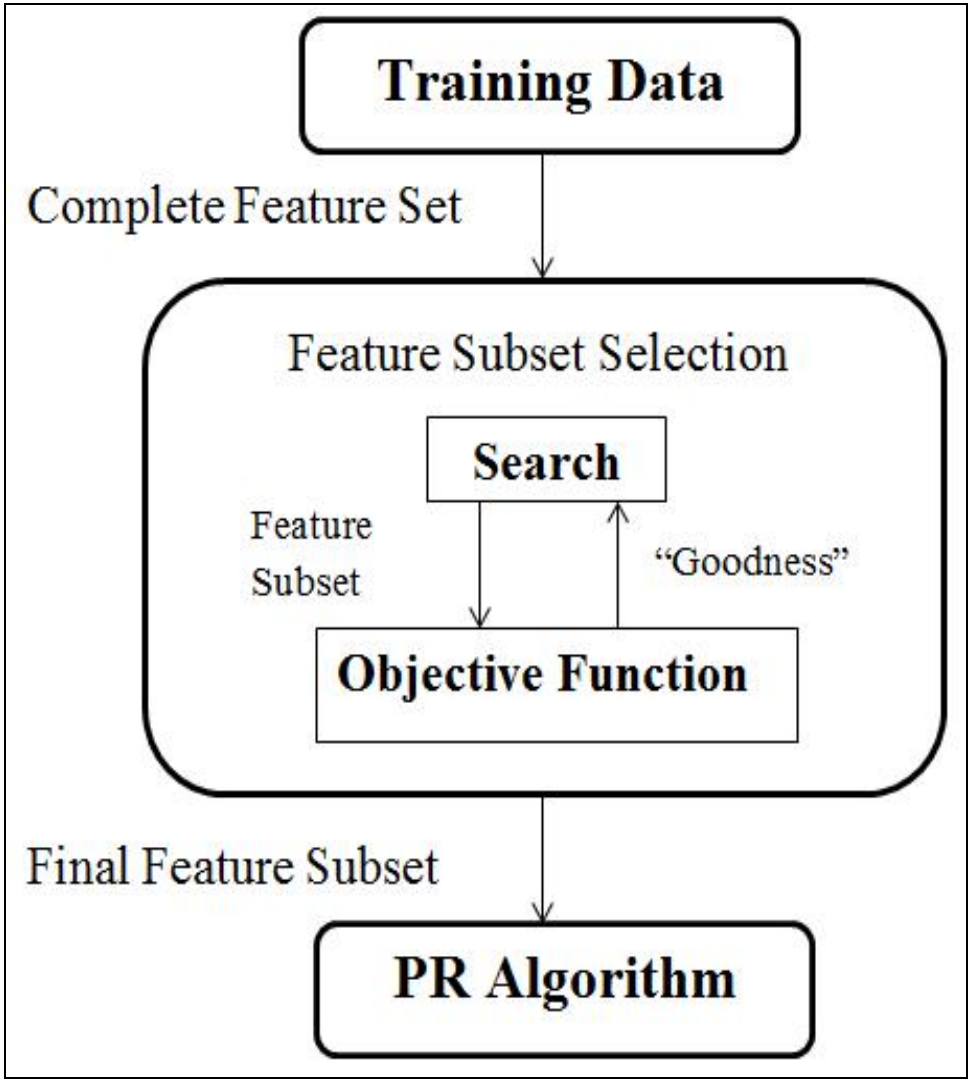


Feature Selection Steps

Feature Selection is an optimization problem.

Step 1 : Search the space of possible feature set.

Step 2 : Pick the subset that is optimal or near optimal with respect to some objective function.



Feature Selection Methods

- Feature Selection Algorithms can be broadly classified into 3 classes :
  - Filter Methods
  - Wrapper Methods
  - Embedded Methods

Filter Methods

- purely Statistical in nature.
- used as a preprocessing step.
- the selection of features is independent of any machine learning algorithms.
- features are selected on the basis of their scores in various statistical tests for their correlation with the outcome variable.

		Feature	
		Categorical	**Continuous**
Response	**Categorical**	Chi-Square	Anova
	Continuous	LDA	Pearson Correlation

- **LDA:** Linear discriminant analysis is used to find a linear combination of features that characterizes or separates two or more classes (or levels) of a categorical variable.
- **ANOVA:** ANOVA stands for Analysis of variance. It is similar to LDA except for the fact that it is operated using one or more categorical independent features and one continuous dependent feature. It provides a statistical test of whether the means of several groups are equal or not.
- **Chi-Square:** It is a dependency test which helps us to determine either there is any kind of dependence among the variables under observation or not.
- **Pearson's Correlation:** It is used as a measure for quantifying linear dependence between two continuous variables X and Y. Its value varies from -1 to +1.

Wrapper Method

- work on the basic principles of Combinatorics.
- the learning algorithm (classifier) itself is used to perform the Feature Selection and select the top features.
- Examples :
  - forward feature selection
  - backward feature elimination
  - recursive feature elimination

- **Forward Selection:** Forward selection is an iterative method in which we start with having no feature in the model. In each iteration, we keep adding the feature which best improves our model till an addition of a new variable does not improve the performance of the model.
- **Backward Elimination:** In backward elimination, we start with all the features and removes the least significant feature at each iteration which improves the performance of the model. We repeat this until no improvement is observed on removal of features.
- **Recursive Feature elimination:** It is a greedy optimization algorithm which aims to find the best performing feature subset. It repeatedly creates models and keeps aside the best or the worst performing feature at each iteration. It constructs the next model with the left features until all the features are exhausted. It then ranks the features based on the order of their elimination.

Embedded Methods

- combine the qualities of filter and wrapper methods.
- dynamic Feature Selection Methods.
- Based on the inferences that we draw from the previous model, we decide to add or remove features from your subset. The problem is essentially reduced to a search problem.

Thank You !

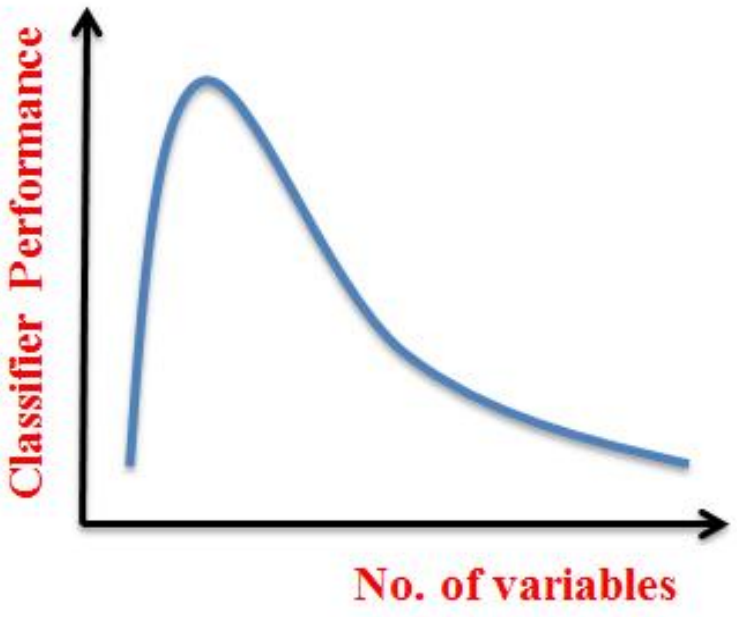
Feature Selection

Feature Selection

- It is the process of selecting a subset of relevant features for use in model construction.
- It is also called variable selection or attribute selection.
- It reduces the complexity of model.
- It either improve or maintain accuracy of model.

Curse of Dimensionality

- The amount of data required to achieve the same level of accuracy increases exponentially as the number of features increases.
- But in practice, the volume of training data available to us is fixed. Therefore, in most cases, the performance of the classifier will decrease with an increased number of variables.

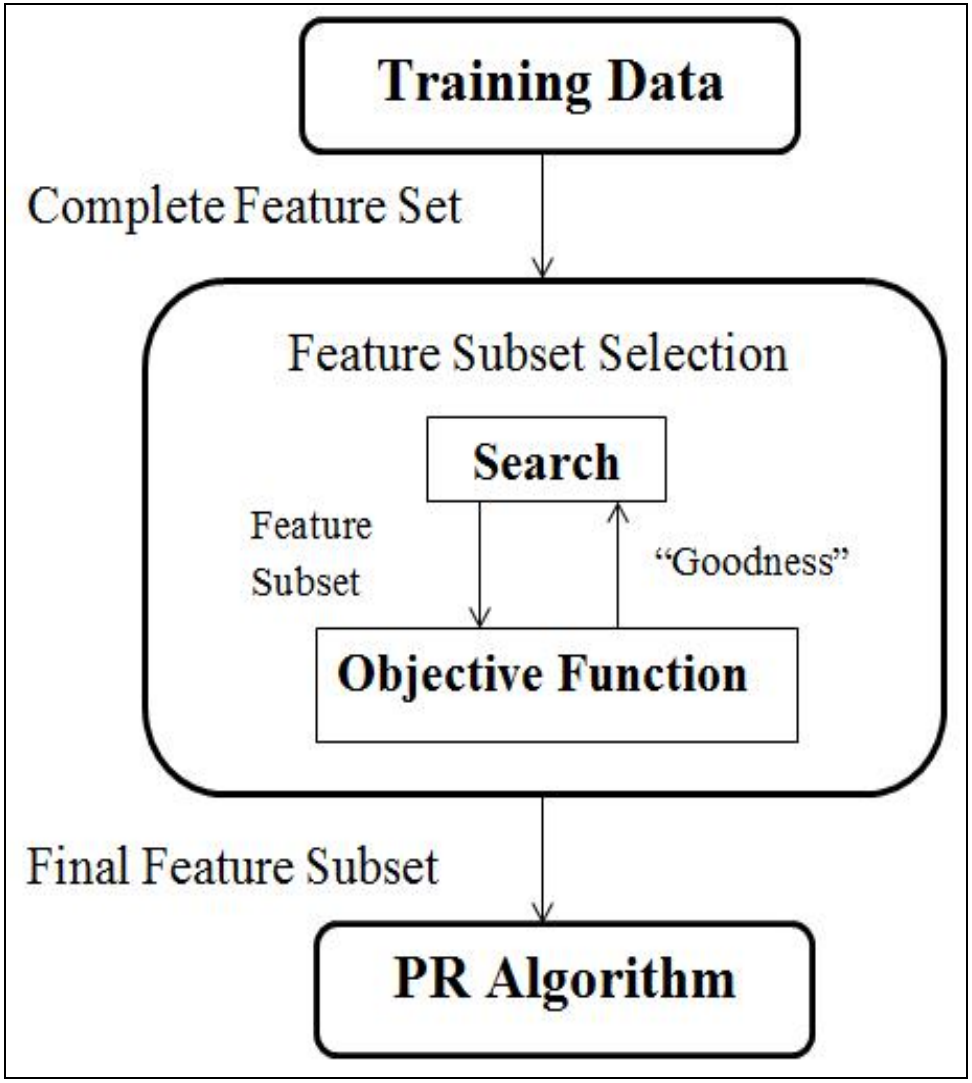


Feature Selection Steps

Feature Selection is an optimization problem.

Step 1 : Search the space of possible feature set.

Step 2 : Pick the subset that is optimal or near optimal with respect to some objective function.



Feature Selection Methods

- Feature Selection Algorithms can be broadly classified into 3 classes :
  - Filter Methods
  - Wrapper Methods
  - Embedded Methods

Filter Methods

- purely Statistical in nature.
- used as a preprocessing step.
- the selection of features is independent of any machine learning algorithms.
- features are selected on the basis of their scores in various statistical tests for their correlation with the outcome variable.

		Feature	
		Categorical	**Continuous**
Response	**Categorical**	Chi-Square	Anova
	Continuous	LDA	Pearson Correlation

- **LDA:** Linear discriminant analysis is used to find a linear combination of features that characterizes or separates two or more classes (or levels) of a categorical variable.
- **ANOVA:** ANOVA stands for Analysis of variance. It is similar to LDA except for the fact that it is operated using one or more categorical independent features and one continuous dependent feature. It provides a statistical test of whether the means of several groups are equal or not.
- **Chi-Square:** It is a dependency test which helps us to determine either there is any kind of dependence among the variables under observation or not.
- **Pearson's Correlation:** It is used as a measure for quantifying linear dependence between two continuous variables X and Y. Its value varies from -1 to +1.

Wrapper Method

- work on the basic principles of Combinatorics.
- the learning algorithm (classifier) itself is used to perform the Feature Selection and select the top features.
- Examples :
  - forward feature selection
  - backward feature elimination
  - recursive feature elimination

- **Forward Selection:** Forward selection is an iterative method in which we start with having no feature in the model. In each iteration, we keep adding the feature which best improves our model till an addition of a new variable does not improve the performance of the model.
- **Backward Elimination:** In backward elimination, we start with all the features and removes the least significant feature at each iteration which improves the performance of the model. We repeat this until no improvement is observed on removal of features.
- **Recursive Feature elimination:** It is a greedy optimization algorithm which aims to find the best performing feature subset. It repeatedly creates models and keeps aside the best or the worst performing feature at each iteration. It constructs the next model with the left features until all the features are exhausted. It then ranks the features based on the order of their elimination.

Embedded Methods

- combine the qualities of filter and wrapper methods.
- dynamic Feature Selection Methods.
- Based on the inferences that we draw from the previous model, we decide to add or remove features from your subset. The problem is essentially reduced to a search problem.

Thank You !

Decision Tree Classification Algorithm

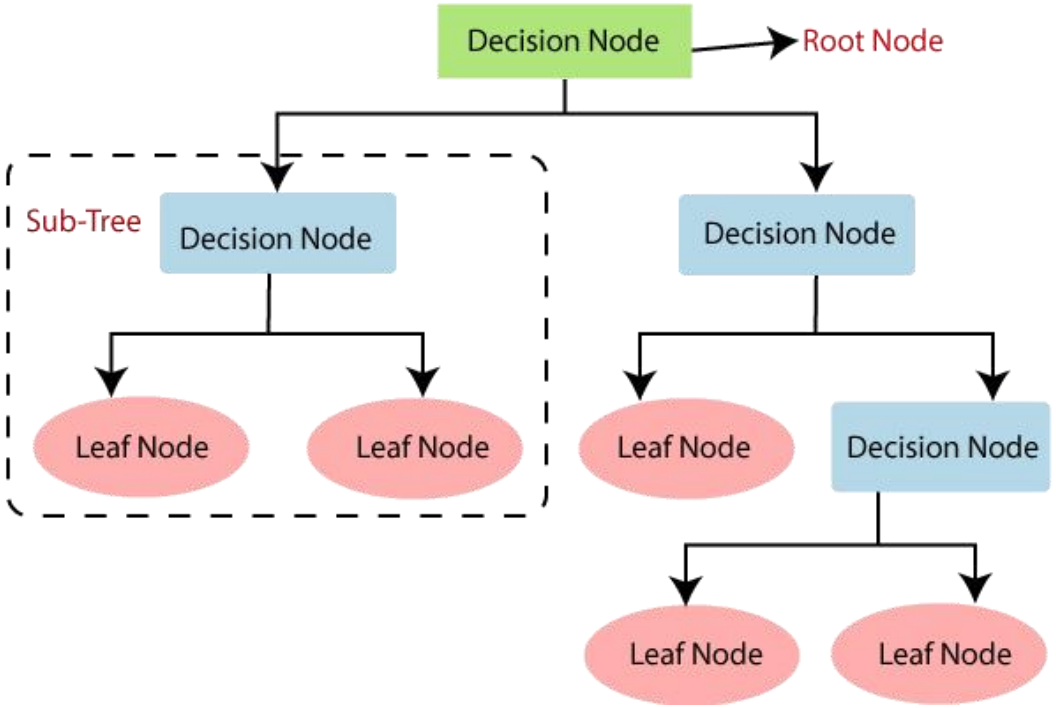
Introduction

- Decision Tree is a **Supervised learning technique** that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems.
- It is a tree-structured classifier, where **internal nodes represent the features of a dataset, branches represent the decision rules** and **each leaf node represents the outcome.**
- In a Decision tree, there are two nodes, which are the **Decision Node** and **Leaf Node.** Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
- ***It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.***

• It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

• In order to build a tree, we use the **CART algorithm,** which stands for **Classification and Regression Tree algorithm.**

• A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.



General structure of a decision tree

Why use Decision Trees?

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.

- The logic behind the decision tree can be easily understood because it shows a tree-like structure.

Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.
- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.
- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.
- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.
- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes.

How does the Decision Tree algorithm Work?

- In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node.

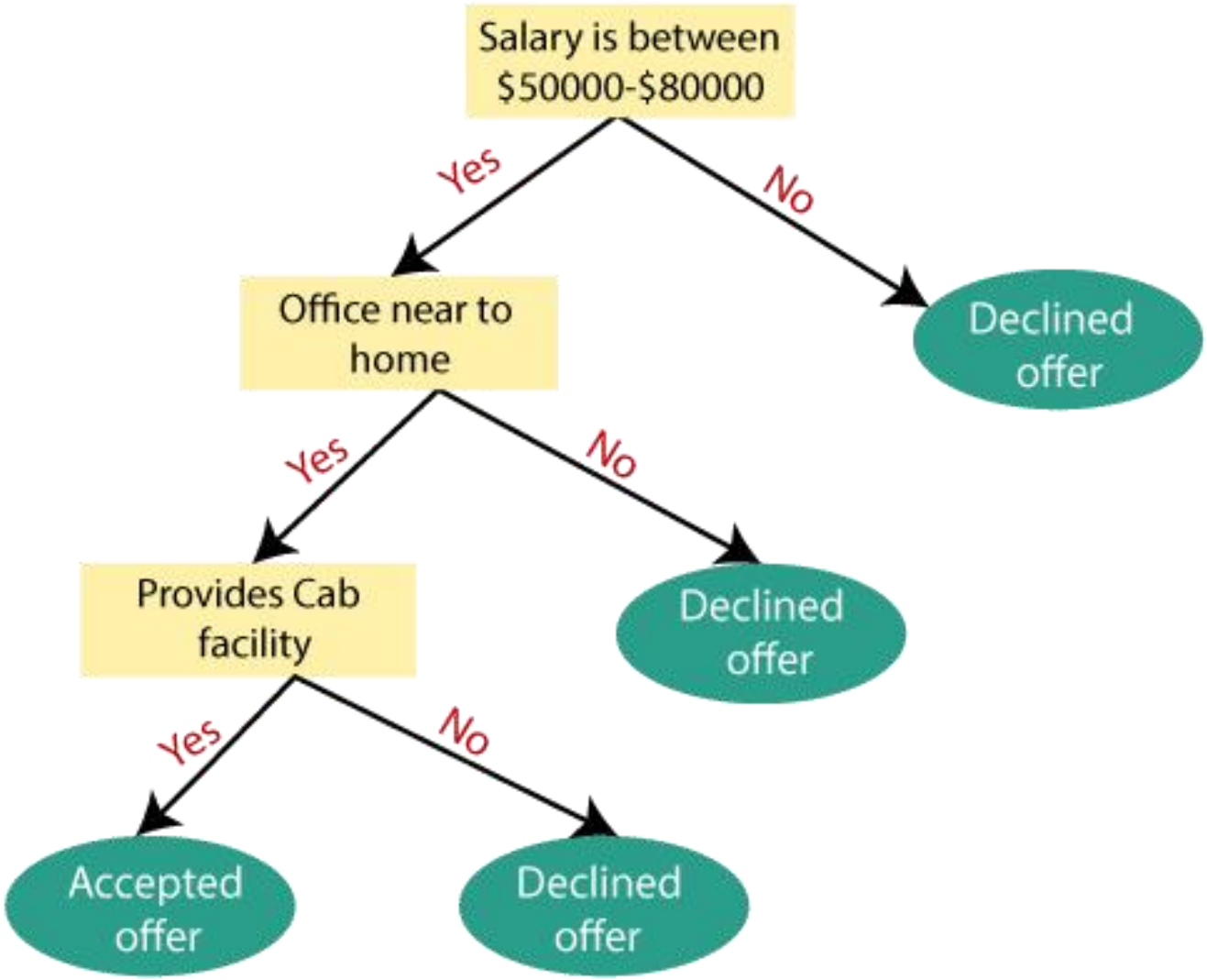
- For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the algorithm.

Algorithm

- **Step-1:** Begin the tree with the root node, says S, which contains the complete dataset.
- **Step-2:** Find the best attribute in the dataset using **Attribute Selection Measure (ASM).**
- **Step-3:** Divide the S into subsets that contains possible values for the best attributes.
- **Step-4:** Generate the decision tree node, which contains the best attribute.
- **Step-5:** Recursively make new decision trees using the subsets of the dataset created in step -3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

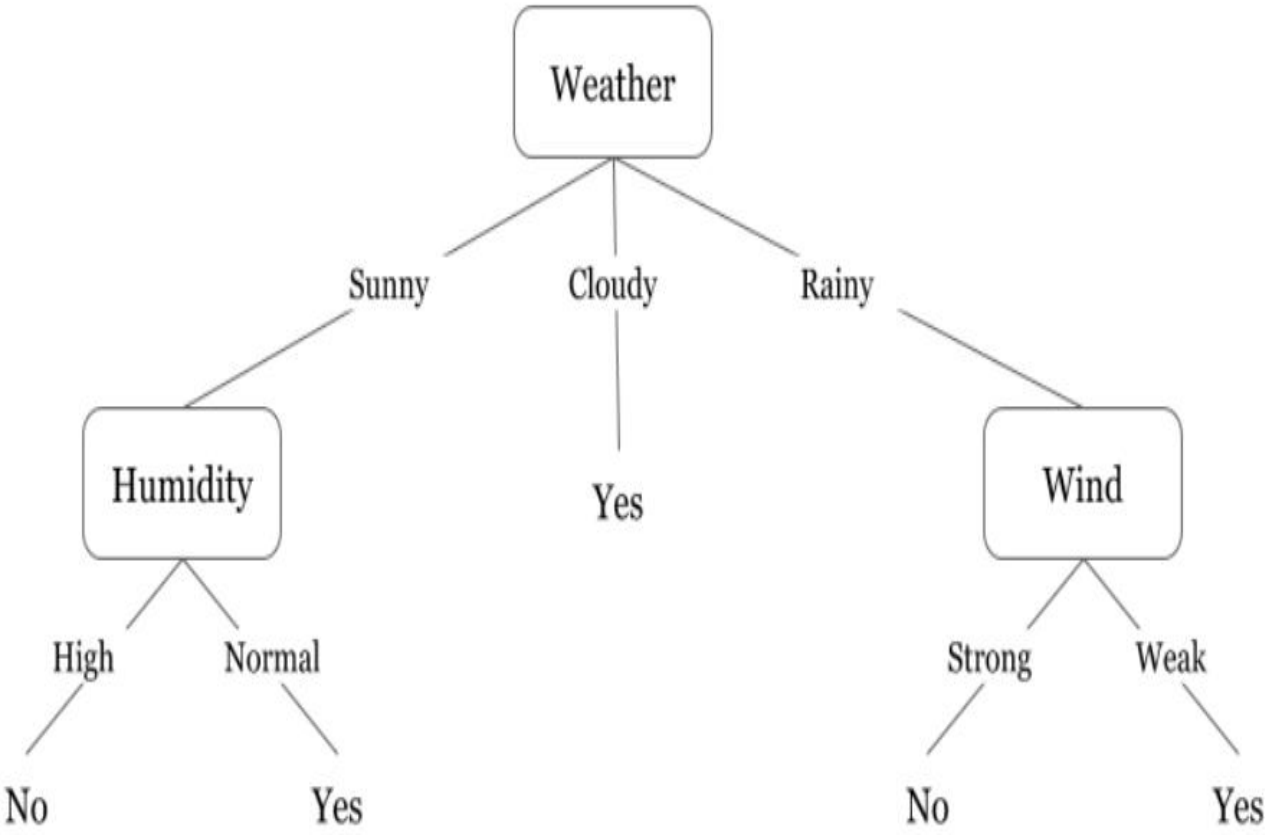
Example

- Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not.
- So, to solve this problem, the decision tree starts with the root node (Salary attribute by ASM).
- The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels.
- The next decision node further gets split into one decision node (Cab facility) and one leaf node.
- Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer).


Salary is between $50000-$80000
Yes
No
Declined offer
Office near to home
Yes
No
Declined offer
Provides Cab facility
Yes
No
Accepted offer
Declined offer

Example of a decision tree

Day	Weather	Temperature	Humidity	Wind	Play?
1	Sunny	Hot	High	Weak	No
2	Cloudy	Hot	High	Weak	Yes
3	Sunny	Mild	Normal	Strong	Yes
4	Cloudy	Mild	High	Strong	Yes
5	Rainy	Mild	High	Strong	No
6	Rainy	Cool	Normal	Strong	No
7	Rainy	Mild	High	Weak	Yes
8	Sunny	Hot	High	Strong	No
9	Cloudy	Hot	Normal	Weak	Yes
10	Rainy	Mild	High	Strong	No



We see that if the *weather is cloudy* then we must go to play. Why didn't it split more? Why did it stop there?
To answer this question, we need to know about few more concepts like entropy, information gain, and Gini index.

Attribute Selection Measures

- While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes.
- So, to solve such problems there is a technique which is called as **Attribute selection measure or ASM.**
-
- By this measurement, we can easily select the best attribute for the nodes of the tree.
- There are two popular techniques for ASM, which are:

1. **Information Gain**
2. **Gini Index**

Information Gain

- Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- It calculates how much information a feature provides us about a class.
- According to the value of information gain, we split the node and build the decision tree.
- A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first.
- It can be calculated using the below formula:

Information Gain= Entropy(S)- [(Weighted Avg) *Entropy(each feature)]

- **Entropy:** Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy(s)} = -P(\text{yes})\log_2 P(\text{yes}) - P(\text{no})\log_2 P(\text{no})$$

Where,

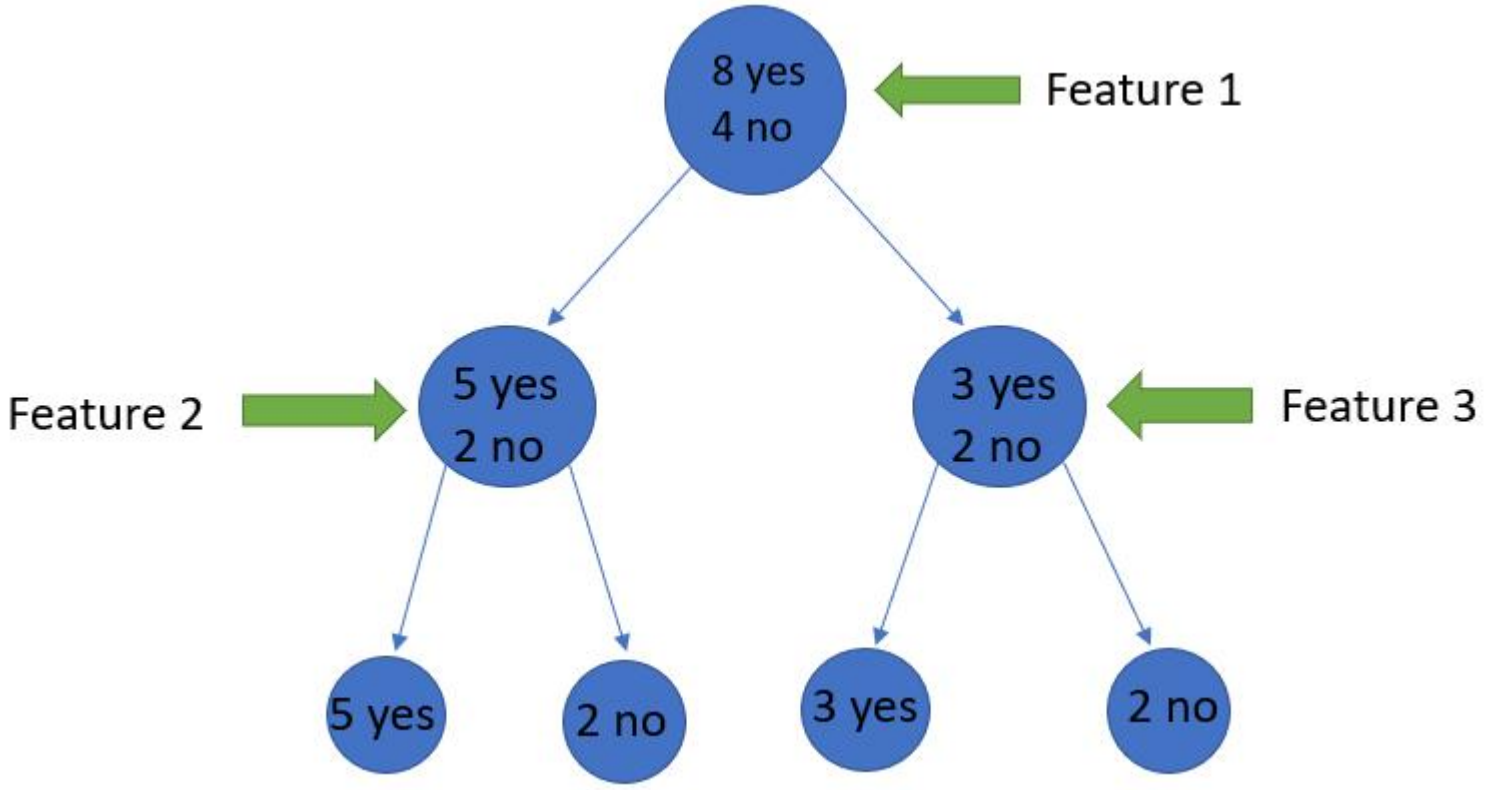
S= Total number of samples
P(yes)= probability of yes
P(no)= probability of no

How to calculate Entropy?

- Suppose a *feature* has 8 “yes” and 4 “no” initially, after the first split the left node *gets 5 ‘yes’ and 2 ‘no’* whereas right node *gets 3 ‘yes’ and 2 ‘no’*.

- We see here the split is not pure, why? Because we can still see some negative classes in both the nodes. In order to make a decision tree, we need to calculate the impurity of each split, and when the purity is 100%, we make it as a leaf node.

- To check the impurity of feature 2 and feature 3 we will take the help for Entropy formula.



We can clearly see from the tree itself that left node has low entropy or more purity than right node since left node has a greater number of "yes" and it is easy to decide here.

Always remember that the **higher the Entropy, the lower will be the purity and the higher will be the impurity.**

For feature 2 :

$$\Rightarrow -\left(\frac{5}{7}\right)log_2\left(\frac{5}{7}\right) - \left(\frac{2}{7}\right)log_2\left(\frac{2}{7}\right)$$
$$\Rightarrow -(0.71 * -0.49) - (0.28 * -1.83)$$
$$\Rightarrow -(-0.34) - (-0.51)$$
$$\Rightarrow 0.34 + 0.51$$
$$\Rightarrow 0.85$$

For feature 3 :

$$\Rightarrow -\left(\frac{3}{5}\right)log_2\left(\frac{3}{5}\right) - \left(\frac{2}{5}\right)log_2\left(\frac{2}{5}\right)$$
$$\Rightarrow -(0.6 * -0.73) - (0.4 * -1.32)$$
$$\Rightarrow -(-0.438) - (-0.528)$$
$$\Rightarrow 0.438 + 0.528$$
$$\Rightarrow 0.966$$

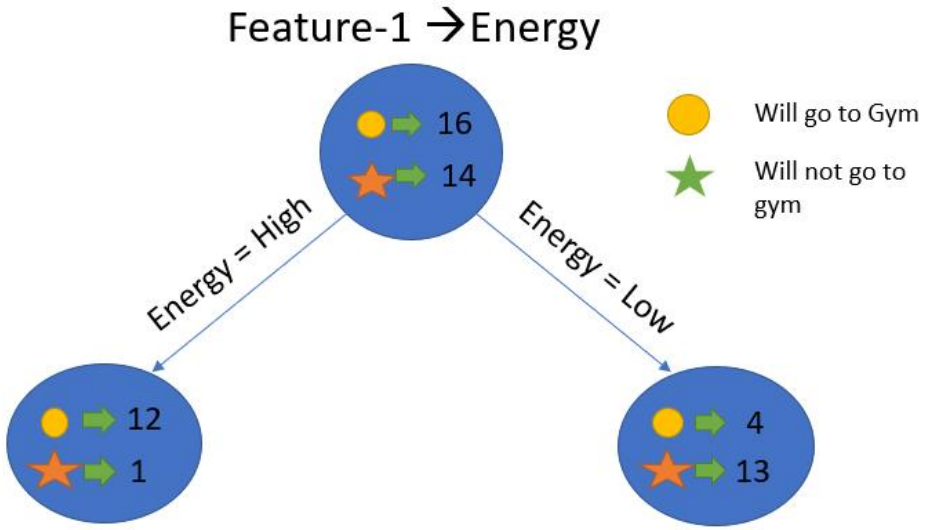
Example to calculate Information Gain

Suppose our entire population has a total of 30 instances. The dataset is to predict whether the person will go to the gym or not.

Let's say 16 people go to the gym and 14 people don't

Now we have two features to predict whether he/she will go to the gym or not.

- Feature 1 is **"Energy"** which takes two values *"high" and "low"*
- Feature 2 is **"Motivation"** which takes 3 values *"No motivation", "Neutral" and "Highly motivated".*



Let's calculate Entropy

$$E(Parent) = -\left(\frac{16}{30}\right)\log_2\left(\frac{16}{30}\right) - \left(\frac{14}{30}\right)\log_2\left(\frac{14}{30}\right) \approx 0.99$$

$$E(Parent|Energy = "high") = -\left(\frac{12}{13}\right)\log_2\left(\frac{12}{13}\right) - \left(\frac{1}{13}\right)\log_2\left(\frac{1}{13}\right) \approx 0.39$$

$$E(Parent|Energy = "low") = -\left(\frac{4}{17}\right)\log_2\left(\frac{4}{17}\right) - \left(\frac{13}{17}\right)\log_2\left(\frac{13}{17}\right) \approx 0.79$$

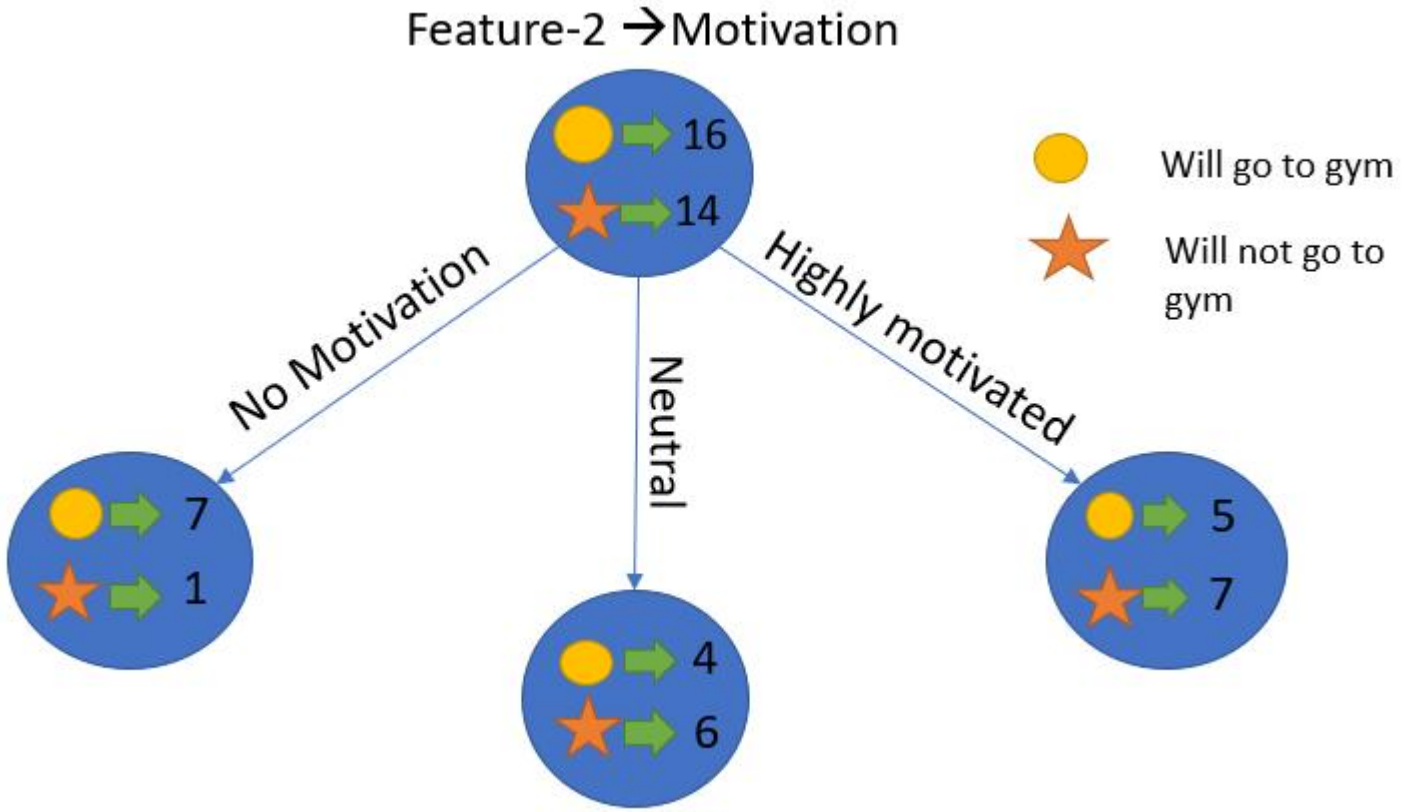
To see the weighted average of entropy of each node we will do as follows:

$$E(Parent|Energy) = \frac{13}{30} * 0.39 + \frac{17}{30} * 0.79 = 0.62$$

Now we have the value of E(Parent) and E(Parent|Energy), information gain will be:

$$\begin{aligned} Information\ Gain &= E(parent) - E(parent|energy) \\ &= 0.99 - 0.62 \\ &= 0.37 \end{aligned}$$

Similarly, we will do this with the other feature “Motivation” and calculate its information gain.



Let's calculate the entropy here:

$$E(Parent) = 0.99$$

$$E\Big(Parent|Motivation = "No\ motivation"\Big) = -\left(\frac{7}{8}\right)log_2\left(\frac{7}{8}\right) - \frac{1}{8}\log_2\left(\frac{1}{8}\right) = 0.54$$

$$E\big(Parent|Motivation = "Neutral"\big) = -\left(\frac{4}{10}\right)log_2\left(\frac{4}{10}\right) - (\frac{6}{10})\log_2\left(\frac{6}{10}\right) = 0.97$$

$$E\big(Parent|Motivation = "Highly\ motivated"\big) = -\left(\frac{5}{12}\right)log_2\left(\frac{5}{12}\right) - (\frac{7}{12})\log_2\left(\frac{7}{12}\right) = 0.98$$

To see the weighted average of entropy of each node we will do as follows:

$$E(Parent|Motivation) = \frac{8}{30} * 0.54 + \frac{10}{30} * 0.97 + \frac{12}{30} * 0.98 = 0.86$$

Now we have the value of E(Parent) and E(Parent|Motivation), information gain will be:

$$\begin{aligned} Information\ Gain &= E(Parent) - E(Parent|Motivation) \\ &= 0.99 - 0.86 \\ &= 0.13 \end{aligned}$$

We now see that the "Energy" feature gives more reduction which is 0.37 than the "Motivation" feature. Hence we will select the feature which has the highest information gain and then split the node based on that feature.

In this example "Energy" will be our root node and we'll do the same for sub-nodes. Here we can see that when the energy is "high" the entropy is low and hence we can say a person will definitely go to the gym if he has high energy, but what if the energy is low? We will again split the node based on the new feature which is "Motivation".

Gini Index

- Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- An attribute with the low Gini index should be preferred as compared to the high Gini index.
- It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- Gini index can be calculated using the below formula:

$$\textbf{Gini Index} = 1 - \sum_j P_j^2$$

When to stop splitting?

Usually, real-world datasets have a large number of features, which will result in a large number of splits, which in turn gives a huge tree. Such trees take time to build and can lead to overfitting. That means the tree will give very good accuracy on the training dataset but will give bad accuracy in test data. There are many ways to tackle this problem through hyperparameter tuning.

1. ***max_depth*** parameter - set the maximum depth of our decision tree. The more the value of ***max_depth***, the more complex your tree will be. Hence you need a value that will not overfit as well as underfit our data.

2. ***min_samples_split*** - set the minimum number of samples for each spilt.

3. ***min_samples_leaf*** – represents the minimum number of samples required to be in the leaf node. The more you increase the number, the more is the possibility of overfitting.

4. ***max_features*** – it helps us decide what number of features to consider when looking for the best split.

Pruning

- ***Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree.***

- A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset.

- Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning.

- There are mainly two types of tree **pruning** technology used:

  Cost Complexity Pruning
  Reduced Error Pruning.

- There are mainly 2 ways for pruning:

  (i) **Pre-pruning** – we can stop growing the tree earlier, which means we can prune/remove/cut a node if it has low importance **while growing** the tree.

  (ii) **Post-pruning** – once our **tree is built to its depth**, we can start pruning the nodes based on their significance.

Advantages of the Decision Tree

- It is simple to understand as it follows the same process which a human follow while making any decision in real-life.
- It can be very useful for solving decision-related problems.
- It helps to think about all the possible outcomes for a problem.
- There is less requirement of data cleaning compared to other algorithms.

Disadvantages of the Decision Tree

- The decision tree contains lots of layers, which makes it complex.
- It may have an overfitting issue, which can be resolved using the **Random Forest algorithm.**
- For more class labels, the computational complexity of the decision tree may increase.

Decision Trees

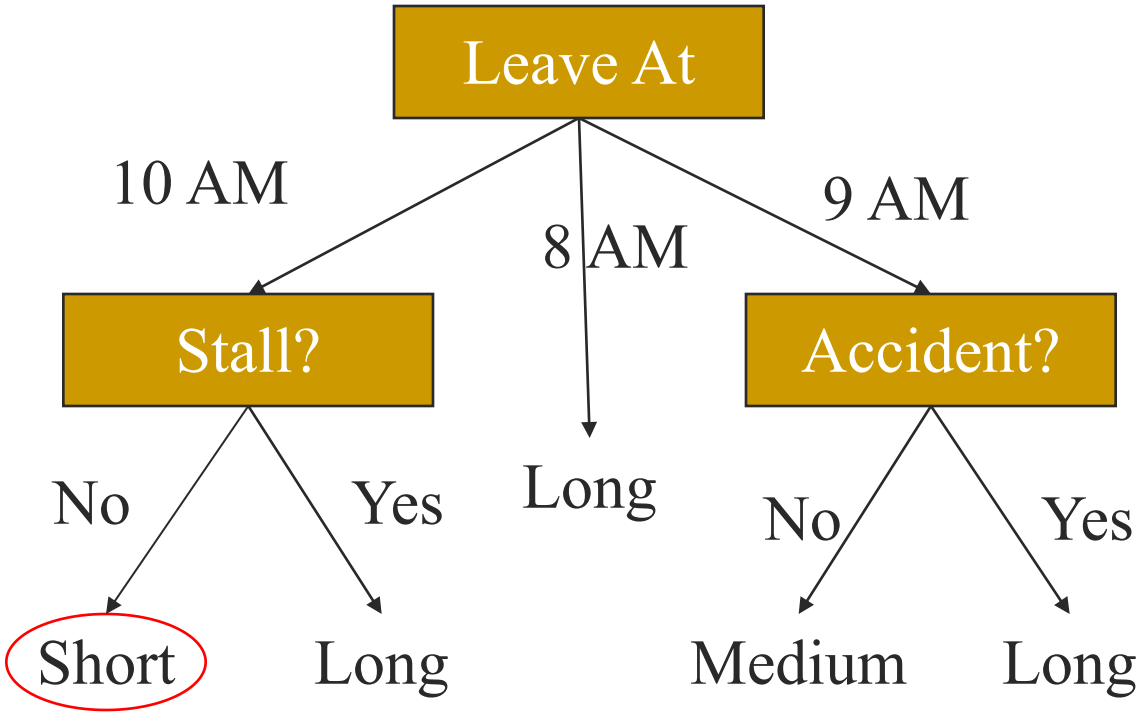
Overview

- What is a Decision Tree
- Sample Decision Trees
- How to Construct a Decision Tree
- Problems with Decision Trees
- Decision Trees in Gaming
- Summary

What is a Decision Tree?

- An *inductive learning task*
  - Use particular facts to make more generalized conclusions
- A predictive model based on a branching series of Boolean tests
  - These smaller Boolean tests are less complex than a one-stage classifier
- Let's look at a sample decision tree…

Predicting Commute Time



If we leave at 10 AM and there are no cars stalled on the road, what will our commute time be?

Inductive Learning

- In this decision tree, we made a series of Boolean decisions and followed the corresponding branch
  - Did we leave at 10 AM?
  - Did a car stall on the road?
  - Is there an accident on the road?
- By answering each of these yes/no questions, we then came to a conclusion on how long our commute might take

Decision Trees as Rules

- We did not have represent this tree graphically
- We could have represented as a set of rules. However, this may be much harder to read…

Decision Tree as a Rule Set

```
if hour == 8am
    commute time = long
else if hour == 9am
    if accident == yes
        commute time = long
    else
        commute time = medium
else if hour == 10am
    if stall == yes
        commute time = long
    else
        commute time = short
```

- Notice that all attributes to not have to be used in each path of the decision.
- As we will see, all attributes may not even appear in the tree.

How to Create a Decision Tree

- We first make a list of attributes that we can measure
  - These attributes (for now) must be discrete
- We then choose a *target attribute* that we want to predict
- Then create an *experience table* that lists what we have seen in the past

Sample Experience Table

Example	Attributes				Target
	Hour	Weather	Accident	Stall	Commute
D1	8 AM	Sunny	No	No	Long
D2	8 AM	Cloudy	No	Yes	Long
D3	10 AM	Sunny	No	No	Short
D4	9 AM	Rainy	Yes	No	Long
D5	9 AM	Sunny	Yes	Yes	Long
D6	10 AM	Sunny	No	No	Short
D7	10 AM	Cloudy	No	No	Short
D8	9 AM	Rainy	No	No	Medium
D9	9 AM	Sunny	Yes	No	Long
D10	10 AM	Cloudy	Yes	Yes	Long
D11	10 AM	Rainy	No	No	Short
D12	8 AM	Cloudy	Yes	No	Long
D13	9 AM	Sunny	No	No	Medium

Choosing Attributes

- The previous experience decision table showed 4 attributes: hour, weather, accident and stall
- But the decision tree only showed 3 attributes: hour, accident and stall
- Why is that?

Choosing Attributes

- Methods for selecting attributes (which will be described later) show that weather is not a discriminating attribute
- We use the principle of *Occam's Razor*: Given a number of competing hypotheses, the simplest one is preferable

Choosing Attributes

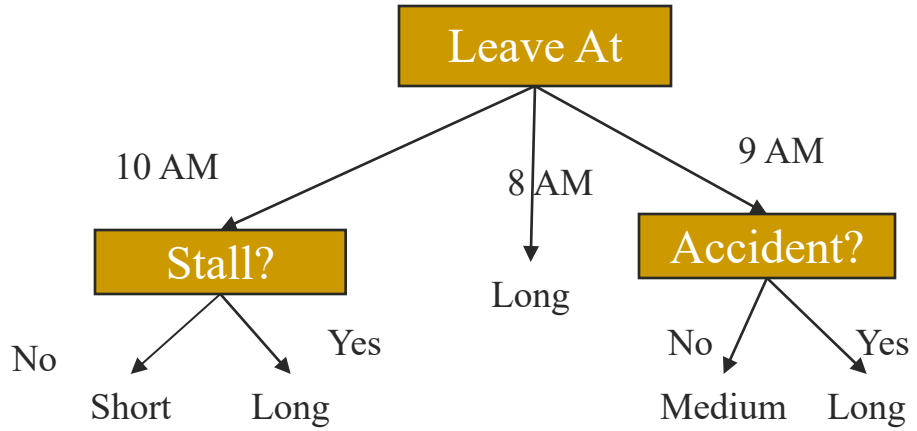
- The basic structure of creating a decision tree is the same for most decision tree algorithms
- The difference lies in how we select the attributes for the tree
- We will focus on the ID3 algorithm developed by Ross Quinlan in 1975

Decision Tree Algorithms

- The basic idea behind any decision tree algorithm is as follows:
  - Choose the *best* attribute(s) to split the remaining instances and make that attribute a decision node
  - Repeat this process for recursively for each child
  - Stop when:
    - All the instances have the same target attribute value
    - There are no more attributes
    - There are no more instances

Identifying the Best Attributes

- Refer back to our original decision tree



- How did we know to split on *leave at* and then on *stall* and *accident* and not weather?

ID3 Heuristic

- To determine the best attribute, we look at the ID3 heuristic
- ID3 splits attributes based on their *entropy*.
- Entropy is the measure of disinformation…

Entropy

- Entropy is minimized when all values of the target attribute are the same.
  - If we know that commute time will always be *short*, then entropy = 0
- Entropy is maximized when there is an equal chance of all values for the target attribute (i.e. the result is random)
  - If commute time = short in 3 instances, medium in 3 instances and long in 3 instances, entropy is maximized

Entropy

- Calculation of entropy
  - Entropy(S) = $\sum_{(i=1 \text{ to } l)} -|S_i|/|S| * \log_2(|S_i|/|S|)$
    - S = set of examples
    - S_i = subset of S with value v_i under the target attribute
    - l = size of the range of the target attribute

ID3

- ID3 splits on attributes with the lowest entropy
- We calculate the entropy for all values of an attribute as the weighted sum of subset entropies as follows:
  - $\sum_{(i = 1 \text{ to } k)} |S_i|/|S|$ Entropy(S_i), where k is the range of the attribute we are testing
- We can also measure information gain (which is inversely proportional to entropy) as follows:
  - Entropy(S) - $\sum_{(i = 1 \text{ to } k)} |S_i|/|S|$ Entropy(S_i)

ID3

- Given our commute time sample set, we can calculate the entropy of each attribute at the root node

Attribute	Expected Entropy	Information Gain
Hour	0.6511	0.768449
Weather	1.28884	0.130719
Accident	0.92307	0.496479
Stall	1.17071	0.248842

Pruning Trees

- There is another technique for reducing the number of attributes used in a tree - *pruning*
- Two types of pruning:
  - Pre-pruning (forward pruning)
  - Post-pruning (backward pruning)

Prepruning

- In prepruning, we decide during the building process when to stop adding attributes (possibly based on their information gain)

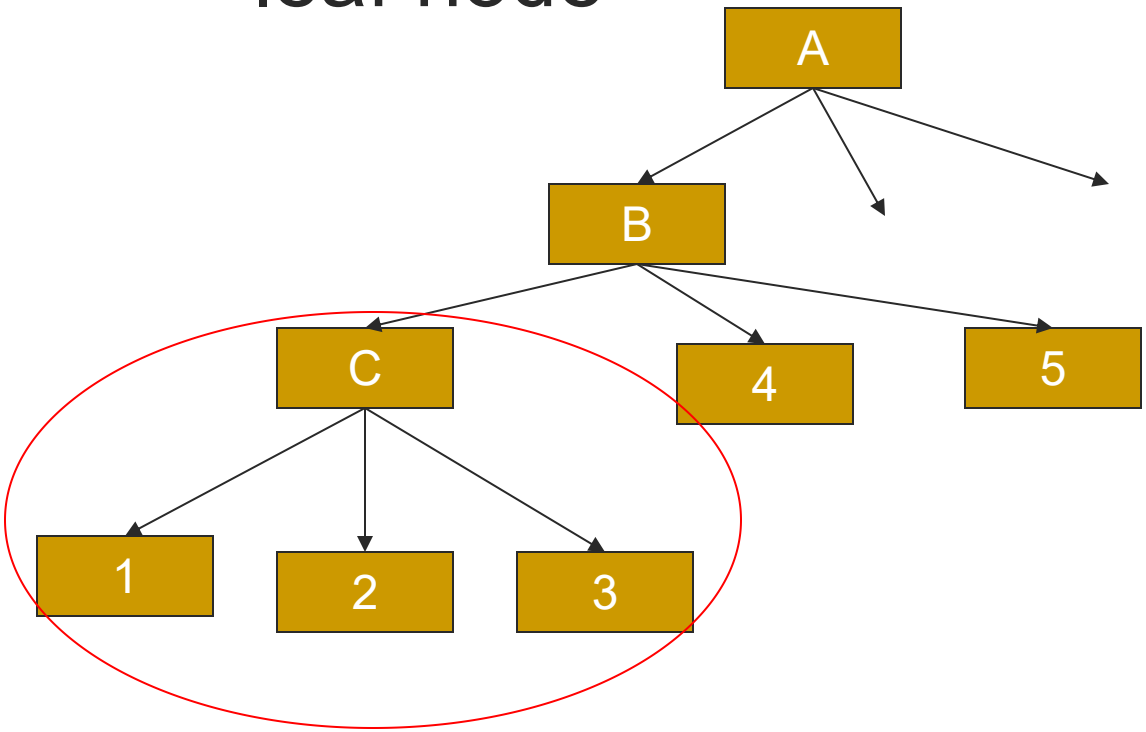
- However, this may be problematic – Why?
  - Sometimes attributes individually do not contribute much to a decision, but combined, they may have a significant impact

Postpruning

- Postpruning waits until the full decision tree has built and then prunes the attributes
- Two techniques:
  - Subtree Replacement
  - Subtree Raising

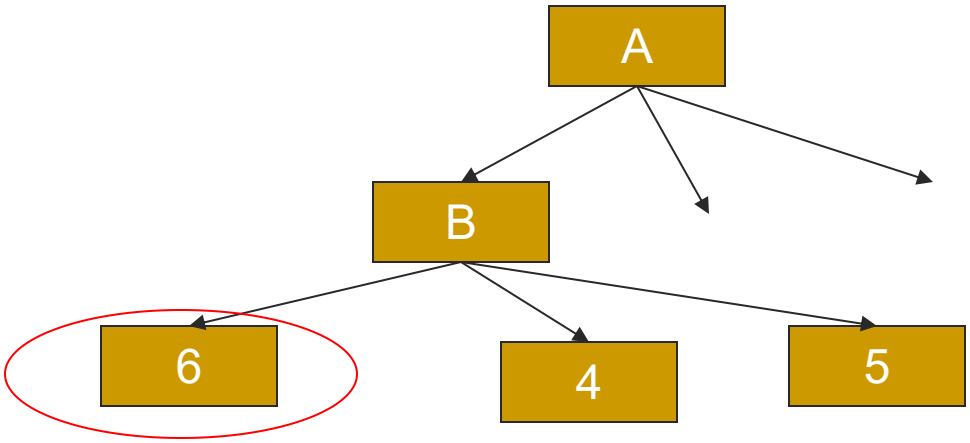
Subtree Replacement

- Entire subtree is replaced by a single leaf node



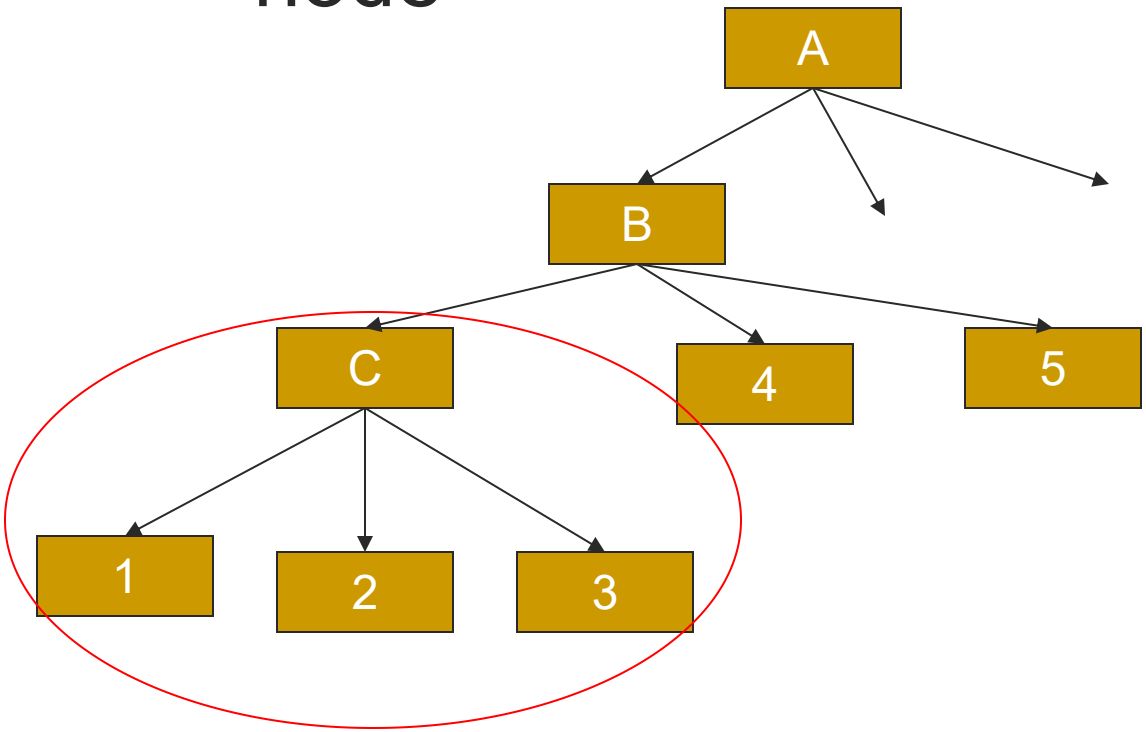
Subtree Replacement

- Node 6 replaced the subtree
- Generalizes tree a little more, but may increase accuracy



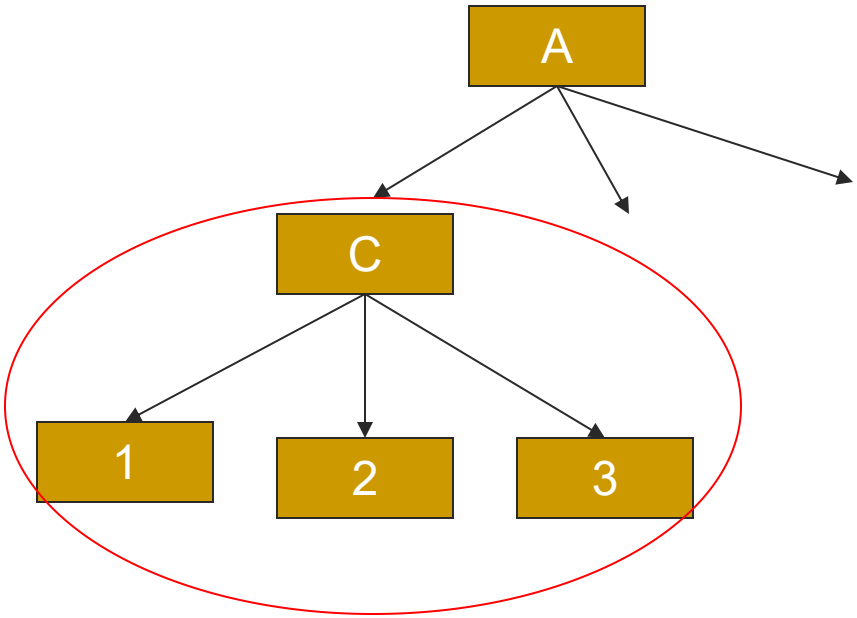
Subtree Raising

- Entire subtree is raised onto another node



Subtree Raising

- Entire subtree is raised onto another node
- This was not discussed in detail as it is not clear whether this is really worthwhile (as it is very time consuming)



Problems with ID3

- ID3 is not optimal
  - Uses *expected* entropy reduction, not actual reduction
- Must use discrete (or discretized) attributes
  - What if we left for work at 9:30 AM?
  - We could break down the attributes into smaller values…

Problems with Decision Trees

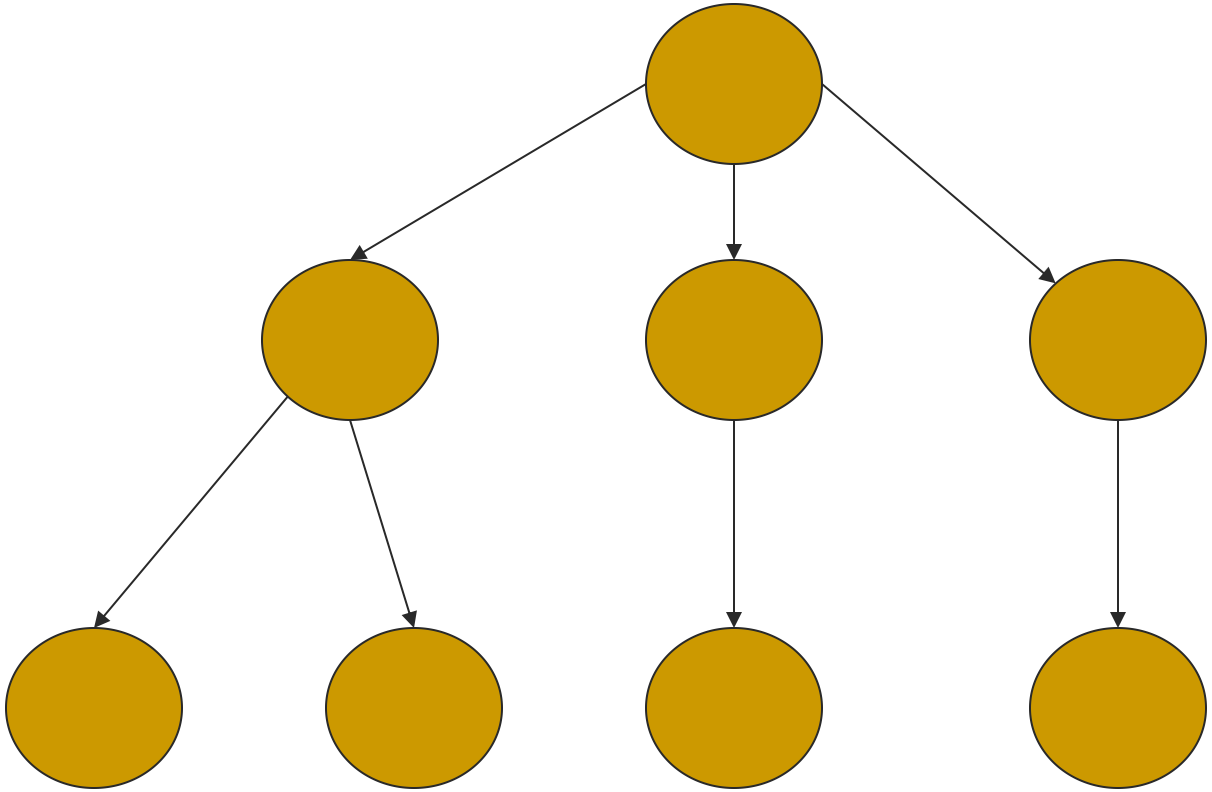
- While decision trees classify quickly, the time for building a tree may be higher than another type of classifier

- Decision trees suffer from a problem of errors propagating throughout a tree
  - A very serious problem as the number of classes increases

Error Propagation

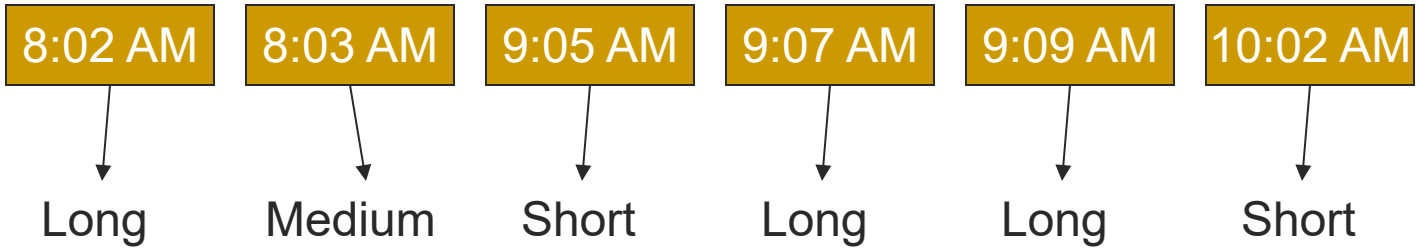
- Since decision trees work by a series of local decisions, what happens when one of these local decisions is wrong?
  - Every decision from that point on may be wrong
  - We may never return to the correct path of the tree

Error Propagation Example



Problems with ID3

- If we broke down leave time to the minute, we might get something like this:



Since entropy is very low for each branch, we have n branches with n leaves. This would not be helpful for predictive modeling.

Problems with ID3

- We can use a technique known as discretization
- We choose *cut points*, such as 9AM for splitting continuous attributes
- These cut points generally lie in a subset of *boundary points*, such that a boundary point is where two adjacent instances in a sorted list have different target value attributes

Problems with ID3

- Consider the attribute commute time

8:00 (L), 8:02 (L), 8:07 (M), 9:00 (S), 9:20 (S), 9:25 (S), 10:00 (S), 10:02 (M)

When we split on these attributes, we increase the entropy so we don't have a decision tree with the same number of cut points as leaves

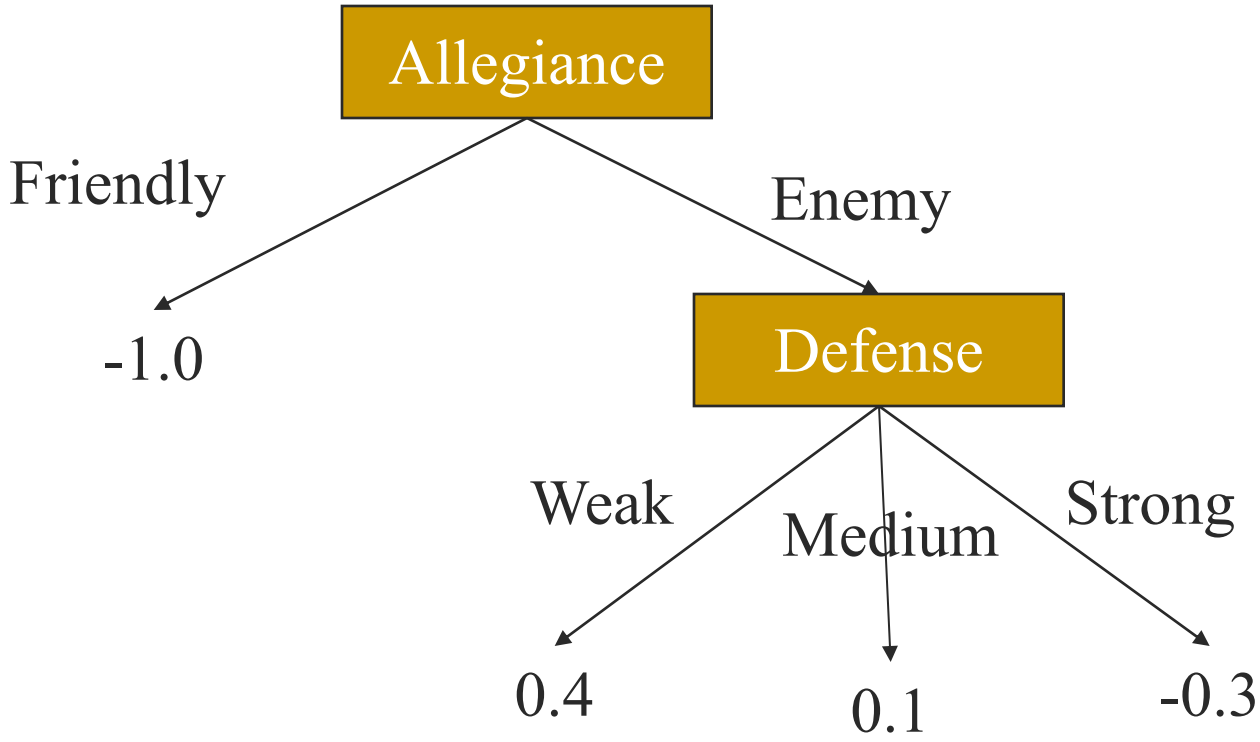
ID3 in Gaming

- Black & White, developed by Lionhead Studios, and released in 2001 used ID3
- Used to predict a player's reaction to a certain creature's action
- In this model, a greater feedback value means the creature should attack

ID3 in Black & White

Example	**Attributes**			**Target**
	Allegiance	Defense	Tribe	Feedback
D1	Friendly	Weak	Celtic	-1.0
D2	Enemy	Weak	Celtic	0.4
D3	Friendly	Strong	Norse	-1.0
D4	Enemy	Strong	Norse	-0.2
D5	Friendly	Weak	Greek	-1.0
D6	Enemy	Medium	Greek	0.2
D7	Enemy	Strong	Greek	-0.4
D8	Enemy	Medium	Aztec	0.0
D9	Friendly	Weak	Aztec	-1.0

ID3 in Black & White



Note that this decision tree does not even use the *tribe* attribute

ID3 in Black & White

- Now suppose we don't want the entire decision tree, but we just want the 2 highest feedback values
- We can create a Boolean expressions, such as

  ((Allegiance = Enemy) ^ (Defense = Weak)) v
  ((Allegiance = Enemy) ^ (Defense = Medium))

Summary

- Decision trees can be used to help predict the future
- The trees are easy to understand
- Decision trees work more efficiently with discrete attributes
- The trees may suffer from error propagation

User Retention

The problems

- Most marketers and product managers only care about growth
- Companies capture an email and stop there.
- Focus too much on lead magnets and opt-ins
- Companies aren’t speaking with their customers
- Companies aren’t listening to their customers
- Startups aren’t building a product that people love
- Loyal customers don’t come overnight, it takes time and a lot of tweaking

Why User Retention?

According to a research report by HBR, “increasing customer retention rates by 5% increases profits by 25% to 95%”.

1. It cost a lot more to acquire a new customer than it does to retain one.
2. A happy customer is more likely to stick around longer and that also means that they’re going to be spending more money with you.
3. A retained happy customer is more likely to refer you to their friends and co-workers, which lowers your cost of growth.
4. A good user retention rate will also mean that your customer life time value will be much higher as well.

Listening to your customers

Listening to your customers is a requirement for building the ultimate product.

Examples:

- Lacey Baier's food blog "A Sweet Pea Chef"
- Neil Patel , Quicksprouts

Two ways of surveying

- Traditional survey
- 1 click survey



YesInsights One-Click Survey

Acme Widgets

Hi Taylor,

You recently purchased a Widget. To help us ensure a great experience for everyone, would you take a moment to rate the quality?

The quality was very good

The quality was acceptable

The quality was unacceptabale

Thank you for your help!

VS



Traditional Survey

Acme Widgets

Hi Taylor,

You recently purchased a Widget. To help us ensure a great experience for everyone, would you take a moment to rate the quality in a short survey?

Take our survey

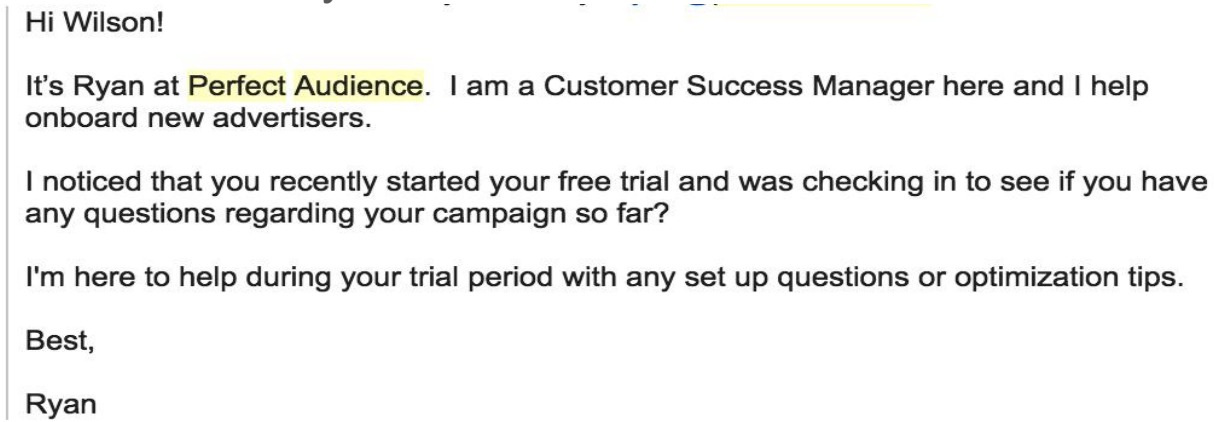
This will take less than a minute of your time.

Thank you for your help!

Engaging with your users

Chat with your users often

It’s important to check in with your customers throughout their lifecycle. You want to see if they are experiencing any roadblocks or if they are running into any feature that they don’t know how to use.



Hi Wilson!

It’s Ryan at Perfect Audience. I am a Customer Success Manager here and I help onboard new advertisers.

I noticed that you recently started your free trial and was checking in to see if you have any questions regarding your campaign so far?

I'm here to help during your trial period with any set up questions or optimization tips.

Best,

Ryan

Ask Questions

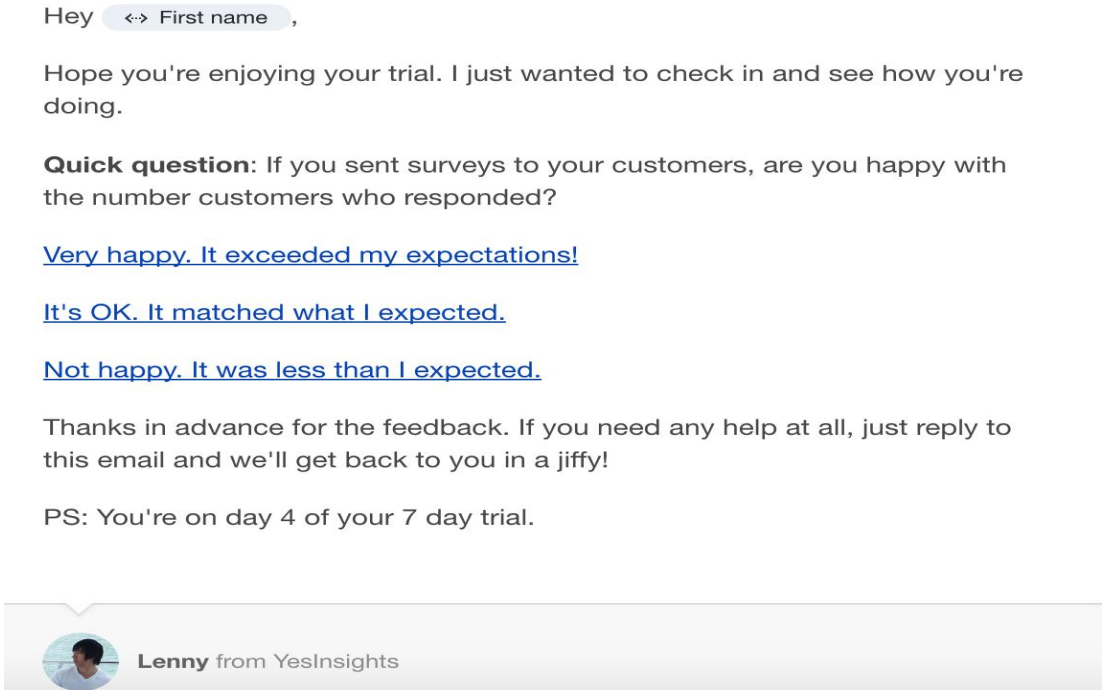
Checking in like how Ryan is doing it works, but sometime you still won't receive a response. To achieve the best results, you should add a combination of things and that includes asking questions.

3 goals:

1. Generate a conversation

2. Actionable feedbacks

3. Remind the user



Provide more educational materials

Hi ↔ First name ! I hope thing are going well so far.

I know you must be thinking about what questions to ask your customers, and how to ask them so you get productive answers. We've thought a lot about that too and put some of our thoughts in this helpful blog post:

http://blog.yesinsights.com/convert-leads-to-paid-customers-with-email-surveys/

This should give you some great ideas to get started.

Cheers!



Lenny from YesInsights

Hey Wilson, Wade here, CEO of Zapier. Our Zapbot tells me that you haven't had much luck setting up a Zap (our word for an automation sequence) yet. How can we help?

Once you get your first Zap set up, it can save you hours of time during the week. Ready to try it out? Here are some of our most popular Zaps:



Save Typeform entries to a Google Sheets spreadsheet — Use This!



Create Trello Cards from Evernote Notes — Use This!



Send your important emails to Slack — Use This!



Turn Trello cards into Google Calendar events — Use This!

Announce new feature updates

Users don’t notice when your product development slows down. However, if things go quiet for long enough, they’ll be easily distracted when a competitor releases a new feature, whether valuable or frivolous.

- Blog
- Email newsletter
- Slack
- Community outreach
- Email outreach to existing customers to surprise them.

Offering discount/referral codes and giveaways

$14,000 in extra revenue plus an upsell.

> Hey Sally! We've been testing out some new pricing models for Baremetrics and instead of our usual 2-months free for switching to annual pricing, we're offering 3-months for free to a subset of customers.
>
> Would you be interested in that?
>
> Reply and let me know and I can add the right coupon to your account and make the switch to annual for you.
>
> Thanks so much for being a part of Baremetrics. It means a lot to have folks like you on board. 🙂

Josh Pigford
Founder of Baremetrics

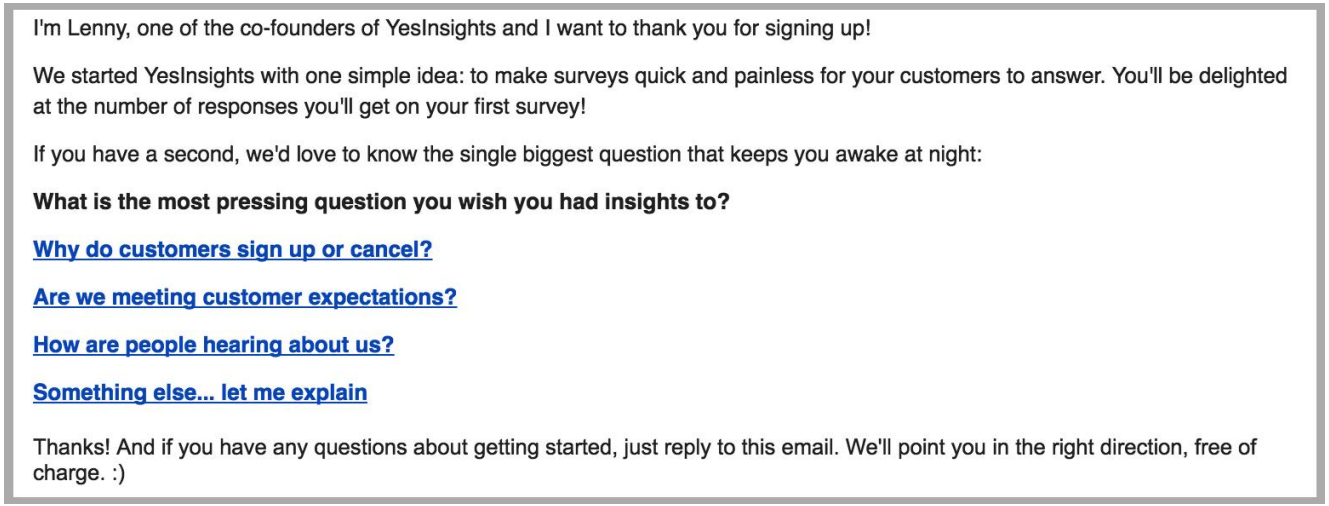
Make a killer first impression

Explain how your app works or the story behind your idea.

Provide them with resources on how to get started.

Let them know that you're here to help and that they can reach out anytime if they ever need help.

Engage in an early conversion or send a warm greeting.



I'm Lenny, one of the co-founders of YesInsights and I want to thank you for signing up!

We started YesInsights with one simple idea: to make surveys quick and painless for your customers to answer. You'll be delighted at the number of responses you'll get on your first survey!

If you have a second, we'd love to know the single biggest question that keeps you awake at night:

What is the most pressing question you wish you had insights to?

Why do customers sign up or cancel?

Are we meeting customer expectations?

How are people hearing about us?

Something else... let me explain

Thanks! And if you have any questions about getting started, just reply to this email. We'll point you in the right direction, free of charge. :)